TECHNICAL DESIGN SPECIFICATION

Carbon-Aware FCFS CPU Scheduler

1. Scope

This document specifies the current FCFS prototype and the interface required to extend the project to additional scheduling algorithms and virtual-machine validation.

2. Process Model

Field	Meaning	Constraint
PID	Process identifier	Unique
AT	Arrival time	AT ≥ 0
BT	CPU burst time	BT > 0
DL	Deadline	DL ≥ AT + BT for feasible case

3. Carbon Trace

The current trace is represented as time–carbon pairs in `carbon.txt`. The scheduler reads the trace and returns the carbon intensity corresponding to a simulated second.

4. Scheduling Policy

- Base order is FCFS.
- At a process boundary, inspect current carbon intensity.
- Only high-carbon periods trigger a forward search.
- Search for a lower-carbon start that still allows full completion before the deadline.
- If none exists, execute immediately.
- Carbon is accumulated over the actual execution interval.

5. Pseudocode

```
CARBON_AWARE_FCFS(processes, carbon_trace):
    sort processes by FCFS order
    time = 0

    for each process p:
        if time < p.arrival:
            time = p.arrival

        if carbon(time) > THRESHOLD:
            candidate = earliest t >= time
                        such that carbon(t) <= THRESHOLD
                        and t + p.burst <= p.deadline
            if candidate exists:
                time = candidate

        p.start = time
        p.finish = time + p.burst
        p.waiting = p.start - p.arrival
        p.turnaround = p.finish - p.arrival
```

```
        time = p.finish

    compute energy and carbon
    return results
```

6. Complexity

The FCFS traversal is O(n). The current forward search for a green time can, in the worst case, inspect a range of simulated time values for each process, giving an approximate O(n·T) behavior where T is the relevant trace horizon. This can be reduced later using indexed green-window structures.

7. Metric Definitions

Waiting Time = Start Time − Arrival Time.

Turnaround Time = Completion Time − Arrival Time.

CPU Utilization = Busy Time / Total Simulated Time × 100.

Energy = Power × Execution Time.

Carbon = time-resolved energy multiplied by the carbon-intensity signal.

8. Extension Interface

Algorithm	Base decision	Carbon layer	Planned status
FCFS	Arrival order	Temporal shift	Implemented
SJF	Shortest burst	Temporal shift + feasibility	Planned
SRTF	Shortest remaining time	Dynamic carbon checks	Planned
Priority	Priority order	Temporal shift + priority	Planned
Round Robin	Time quantum	Carbon-aware quantum/start policy	Planned

9. VM Validation Architecture

The planned VM stage will place the scheduler in a controlled Linux virtual machine environment. The VM will provide repeatable CPU and workload conditions while the host-side experiment harness records execution time, CPU utilization and, where available, power-related telemetry. The simulation and VM layers should share the same workload schema and carbon-trace interface so that algorithmic and systems-level results remain comparable.

10. Implementation Artifact Structure

`fcfs.c` — conventional baseline scheduler.

`carbon_fcfs.c` — carbon-aware FCFS scheduler.

`carbon.txt` — carbon-intensity trace.

`data/` — workload definitions.

`results/` — machine-readable outputs.

`graphs/` — generated figures.

Python automation scripts — workload execution, parsing, analysis and figure generation.